

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Ulepszony procesor 2

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 13

Studia dzienne

Informatyka

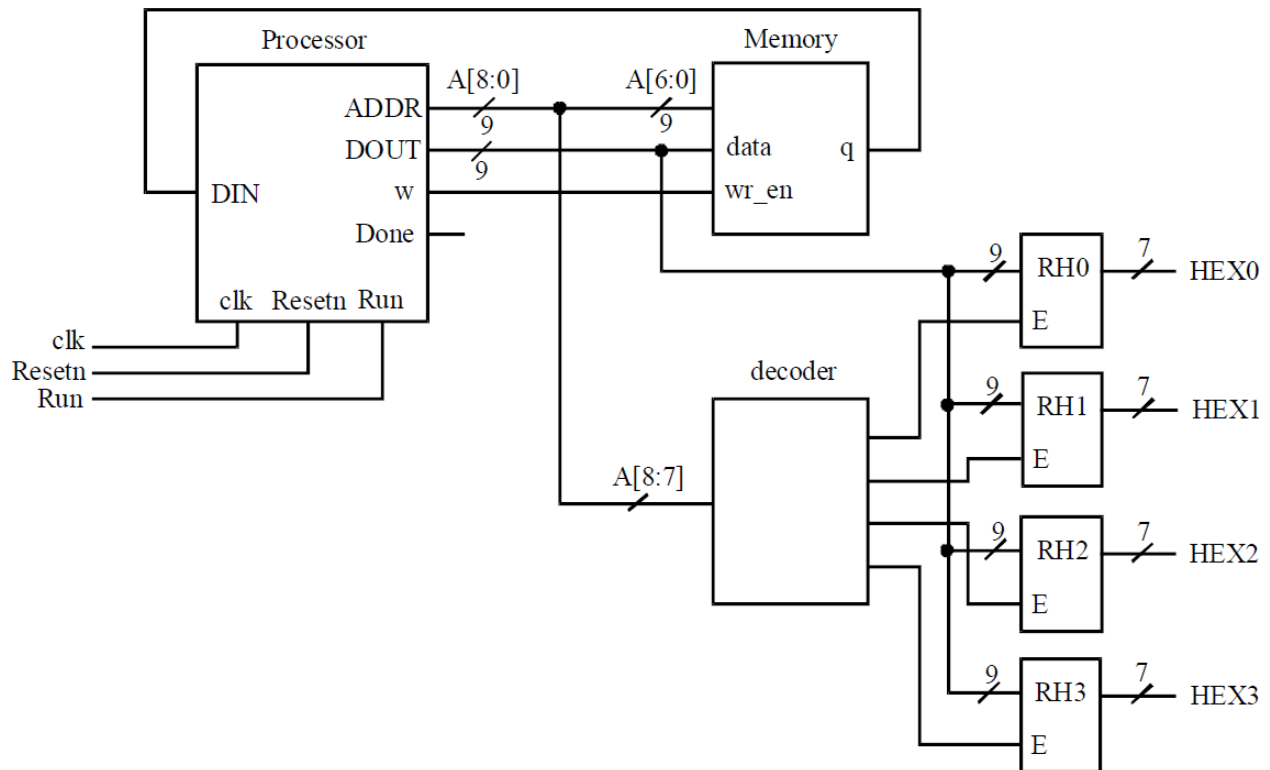
Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salauyou

Data: 09.06.2022

Zadanie 1

Utwórz projekt **processor_with_hex**, którego strukturę pokazano na rys. 4.



Rys. 4. Podłączenie do procesora wyświetlaczy **HEX0-3**

Tutaj linie magistrali adresów A [6: 0] są używane do adresowania lokalizacji pamięci, a A [8:7] jest używany do adresowania zewnętrznych rejestrów RH0-3. Wyjścia rejestrów bezpośrednio sterują 7-segmentowymi wyświetlaczami HEX0-HEX3 na płycie.

Napisz program dla procesora, który wyświetla słowo na wyświetlaczach, przewija słowa w prawo i w lewo. Symuluj projekt i testuj projekt na płycie.

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module processor_with_hex_on_board(
    input [9:9] SW,
    input [0:0] KEY,
    input CLOCK_50,
    output [6:0] HEX0, HEX1, HEX2, HEX3);

    processor_with_hex proc(KEY[0], CLOCK_50, SW[9], HEX0, HEX1, HEX2, HEX3);

endmodule
```

Moduł procesora z pamięcią RAM i zewnętrznymi rejestrami wyjściowymi

```
module processor_with_hex(
    input Resetn, Clock, ext_Run,
```

```

output [6:0] RH0_Q, RH1_Q, RH2_Q, RH3_Q);

wire [8:0] q;
wire [8:0] DIN = q, ADDR, DOUT /* synthesis keep */;
wire wr_en /* synthesis keep */;
RAM128x9 ram(ADDR[6:0], Clock, DOUT, wr_en, q);
reg T1, T2 /* synthesis keep */;
always @ (posedge Clock) begin
    T2 <= T1;
    T1 <= ext_Run;
end
wire Run = T2 /* synthesis keep */;
processor_2 proc(DIN, Resetn, Clock, Run, ADDR, DOUT, wr_en);
// dekodér
wire [3:0] RH_E;
decoder2_4 dec(ADDR[8:7], wr_en, RH_E);
regn # (7) RH0(DOUT[6:0], RH_E[0], Clock, RH0_Q);
regn # (7) RH1(DOUT[6:0], RH_E[1], Clock, RH1_Q);
regn # (7) RH2(DOUT[6:0], RH_E[2], Clock, RH2_Q);
regn # (7) RH3(DOUT[6:0], RH_E[3], Clock, RH3_Q);

endmodule

```

Moduł dekodera z 2-bitowego kodu binarnego na 4-bitowy kod one-hot

```

module decoder2_4(
    input [1:0] s,
    input enable,
    output [3:0] out);

    assign out[0] = (~s[1]) & (~s[0]) & enable;
    assign out[1] = (~s[1]) & s[0] & enable;
    assign out[2] = s[1] & (~s[0]) & enable;
    assign out[3] = s[1] & s[0] & enable;

endmodule

```

Moduł procesora

```

module processor_2(
    input [8:0] DIN,
    input Resetn, Clock, Run,
    output [8:0] ADDR, DOUT,
    output W);

    // stany FSM
    localparam [2:0] T0 = 3'b000, T1 = 3'b001, T2 = 3'b010, T3 = 3'b011,
        T4 = 3'b100, T5 = 3'b101;
    // kody instrukcji
    localparam [2:0] mv = 3'b000, mvi = 3'b001, add = 3'b010, sub = 3'b011,
        ld = 3'b100, st = 3'b101, mvnz = 3'b110;
    reg Done /* synthesis keep */;
    reg [8:0] BusWires /* synthesis keep */;
    wire [0:7] Xreg, Yreg;
    wire [2:0] I;
    wire [9:1] IR;
    wire [8:0] A, G;
    wire [8:0] R0, R1, R2, R3, R4, R5, R6, PC;
    reg G_in, G_out, DIN_out, AddSub, IR_in, A_in;
    wire [8:0] Sum;
    reg [2:0] Tstep_Q /* synthesis keep */;
    reg [2:0] Tstep_D;
    reg [0:7] R_in, R_out;

```

```

reg incr_PC;
wire zero_G = (G == 9'b000000000) /* synthesis keep */;
reg ADDR_in, DOUT_in;
reg W_D;

assign I = IR[9:7];
dec3to8 decX(IR[6:4], 1'b1, Xreg);
dec3to8 decY(IR[3:1], 1'b1, Yreg);

// sterowanie przejściami między stanami FSM
always @ (Tstep_Q, Run, Done)
    case (Tstep_Q)
        T0:
            if (~Run) Tstep_D = T0;
            else Tstep_D = T1;
        T1:
            if (Done) Tstep_D = T0;
            else Tstep_D = T2;
        T2:
            if (Done) Tstep_D = T0;
            else Tstep_D = T3;
        T3:
            if (Done) Tstep_D = T0;
            else Tstep_D = T4;
        T4:
            if (Done) Tstep_D = T0;
            else Tstep_D = T5;
        default: // T5
            Tstep_D = T0;
    endcase

// sterowanie wyjściami FSM
always @ (Tstep_Q, I, Xreg, Yreg, zero_G) begin
    Done = 1'b0;
    G_in = 1'b0;
    G_out = 1'b0;
    A_in = 1'b0;
    AddSub = 1'b0;
    DIN_out = 1'b0;
    R_in = 8'b0000_0000;
    R_out = 8'b0000_0000;
    IR_in = 1'b0;
    incr_PC = 1'b0;
    ADDR_in = 1'b0;
    DOUT_in = 1'b0;
    W_D = 1'b0;
    case (Tstep_Q)
        T0: begin // wyjścia z FSM w takcie 0
            R_out[7] = 1'b1;
            ADDR_in = 1'b1;
        end
        // na zboczu T0 -> T1 zegara zlecamy RAMowi wystawienie kodu instrukcji
        T1: begin // wyjścia z FSM w takcie 1
            // RAM dopiero na zboczu T1 -> T2 będzie miał gotowe odczytane słowo
            incr_PC = 1'b1;
        end
        // na zboczu T1 -> T2 RAM inicjuje podanie odczytanego słowa na DIN
        T2: begin // wyjścia z FSM w takcie 2
            // w wyniku opóźnień chwilę po zboczu T1 -> T2 (ale przed T2 -> T3) na DIN trafia
słowo
            IR_in = 1'b1;
        end
    endcase
end

```

```

// na zboczu T2 -> T3 procesor inicjuje zapisanie w IR słowa z DIN
T3: // wyjścia z FSM w takcie 3
// w wyniku opóźnień chwilę po zboczu T2 -> T3 (ale przed T3 -> T4) do IR trafia kod
instrukcji
    case (I)
        mv: begin
            R_out = Yreg;
            R_in = Xreg;
            Done = 1'b1;
        end
        mvi: begin
            R_out[7] = 1'b1;
            ADDR_in = 1'b1;
            incr_PC = 1'b1;
        end
        add, sub: begin
            R_out = Xreg;
            A_in = 1'b1;
        end
        ld, st: begin
            R_out = Yreg;
            ADDR_in = 1'b1;
        end
        mvnz: begin
            Done = 1'b1;
            if (~zero_G) begin
                R_out = Yreg;
                R_in = Xreg;
            end
        end
    endcase
// w instrukcjach mvi i ld na zboczu T3 -> T4 zlecamy RAMowi wystawienie słowa danych
T4: // wyjścia z FSM w takcie 4
    case (I)
        // mvi, ld: RAM dopiero na zboczu T4 -> T5 będzie miał gotowe odczytane słowo
        add: begin
            R_out = Yreg;
            G_in = 1'b1;
        end
        sub: begin
            R_out = Yreg;
            G_in = 1'b1;
            AddSub = 1'b1;
        end
        st: begin
            R_out = Xreg;
            DOUT_in = 1'b1;
            W_D = 1'b1;
            Done = 1'b1;
        end
    endcase
// w instrukcjach mvi i ld na zboczu T4 -> T5 RAM inicjuje podanie odczytanego słowa na
DIN
T5: // wyjścia z FSM w takcie 5
    case (I)
        mvi, ld: begin
            // w wyniku opóźnień chwilę po zboczu T4 -> T5 (ale przed T5 -> T0) na DIN
            trafia słowo
                DIN_out = 1'b1;
                R_in = Xreg;
                Done = 1'b1;
        end
    end

```

```

        add, sub: begin
            G_out = 1'b1;
            R_in = Xreg;
            Done = 1'b1;
        end
    endcase
endcase
endcase
end

// sterowanie przerzutnikami FSM
always @ (posedge Clock, negedge Resetn)
    if (~Resetn) Tstep_Q <= T0;
    else Tstep_Q <= Tstep_D;

// rejestr
regn #(9) reg_0(BusWires, R_in[0], Clock, R0);
regn #(9) reg_1(BusWires, R_in[1], Clock, R1);
regn #(9) reg_2(BusWires, R_in[2], Clock, R2);
regn #(9) reg_3(BusWires, R_in[3], Clock, R3);
regn #(9) reg_4(BusWires, R_in[4], Clock, R4);
regn #(9) reg_5(BusWires, R_in[5], Clock, R5);
regn #(9) reg_6(BusWires, R_in[6], Clock, R6);
counter #(9) cnt_pc(Resetn, incr_PC, R_in[7], Clock, BusWires, PC);
regn #(9) reg_A(BusWires, A_in, Clock, A);
regn #(9) reg_G(Sum, G_in, Clock, G);
regn #(9) reg_IR(DIN, IR_in, Clock, IR);
regn #(9) reg_ADDR(BusWires, ADDR_in, Clock, ADDR);
regn #(9) reg_DOUT(BusWires, DOUT_in, Clock, DOUT);
regn #(1) reg_W(W_D, 1'b1, Clock, W);

// alu
ripple_carry_adder_subtractor #(9) alu(A, BusWires, AddSub, Sum);

// multiplekser
wire [0:9] mux_s;
assign mux_s = {DIN_out, R_out, G_out};
always @ (*)
    case (mux_s)
        default: BusWires = DIN;
        10'b0100000000: BusWires = R0;
        10'b0010000000: BusWires = R1;
        10'b0001000000: BusWires = R2;
        10'b0000100000: BusWires = R3;
        10'b0000010000: BusWires = R4;
        10'b0000001000: BusWires = R5;
        10'b0000000100: BusWires = R6;
        10'b0000000010: BusWires = PC;
        10'b0000000001: BusWires = G;
    endcase
endmodule

```

Moduł N-bitowego rejestru

```

module regn
    #(parameter N = 9) (
        input [N-1:0] R,
        input Rin, Clock,
        output reg [N-1:0] Q);

    initial Q = {N{1'b0}};
    always @ (posedge Clock)

```

```

        if (Rin) Q <= R;
        else Q <= Q;

endmodule

```

Moduł dekodera z kodu binarnego na 8-bitowy kod one-hot

```

module dec3to8(
    input [2:0] W,
    input En,
    output reg [0:7] Y);

always @ (W, En)
    if (En == 1)
        case (W)
            3'b000: Y = 8'b10000000;
            3'b001: Y = 8'b01000000;
            3'b010: Y = 8'b00100000;
            3'b011: Y = 8'b00010000;
            3'b100: Y = 8'b00001000;
            3'b101: Y = 8'b00000100;
            3'b110: Y = 8'b00000010;
            3'b111: Y = 8'b00000001;
        endcase
    else
        Y = 8'b00000000;

endmodule

```

Moduł sumatora-subtraktora, który używamy jako jednostka arytmetyczno-logiczna (ALU)

```

module ripple_carry_adder_subtractor
    #(parameter N = 4) (
        input [N-1:0] A, B,
        input sub,
        output [N-1:0] S);
    // output cout,
    // output overflow);

    wire [N-2:0] c;
    // assign overflow = c[N-2] ^ cout;
    generate
        genvar i;
        for (i = 0; i < N; i = i + 1)
            begin: ad
                case (i)
                    0: full_adder fa(A[i], sub ^ B[i], sub, S[i], c[i]);
                    N-1: full_adder fa(A[i], sub ^ B[i], c[i-1], S[i], /*cout*/);
                    default: full_adder fa(A[i], sub ^ B[i], c[i-1], S[i], c[i]);
                endcase
            end
        endgenerate

endmodule

```

Moduł pełnego sumatora

```

module full_adder(
    input a, b, cin,
    output s, cout);

    assign s = cin ^ (a ^ b);
    assign cout = a & b | (a ^ b) & cin;

```

```
endmodule
```

Moduł N-bitowego licznika

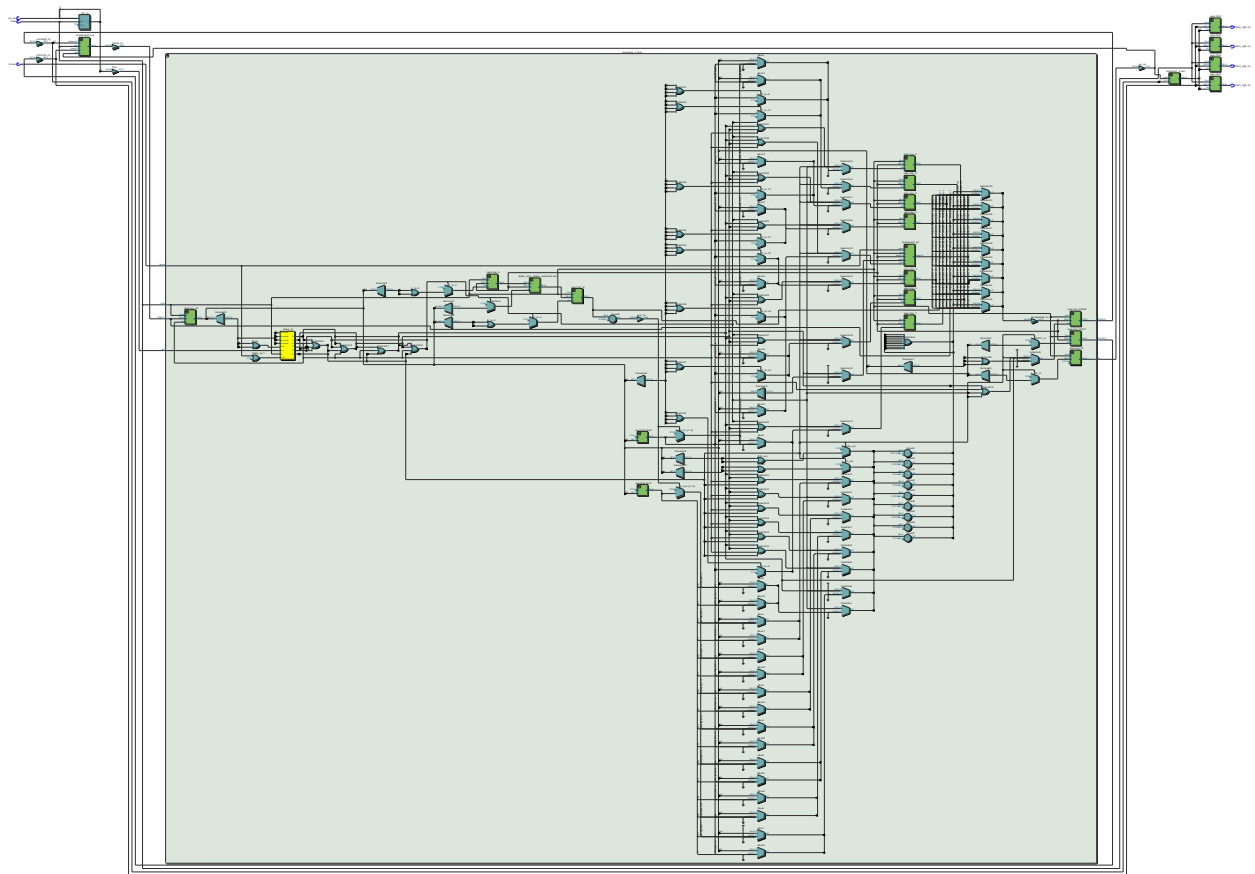
```
module counter
  #(parameter N = 4) (
    input areset, enable, sload, clock,
    input [N-1:0] data,
    output reg [N-1:0] state);

  initial state = {N{1'b0}};
  always @ (posedge clock, negedge areset)
    if (~areset) state = {N{1'b0}};
    else if (sload) state <= data;
    else if (enable) state <= state + 1'b1;
    else state <= state;

endmodule
```

Wyniki syntezy

RTL Viewer



State Machine Viewer układu sterowania

Graf stanów

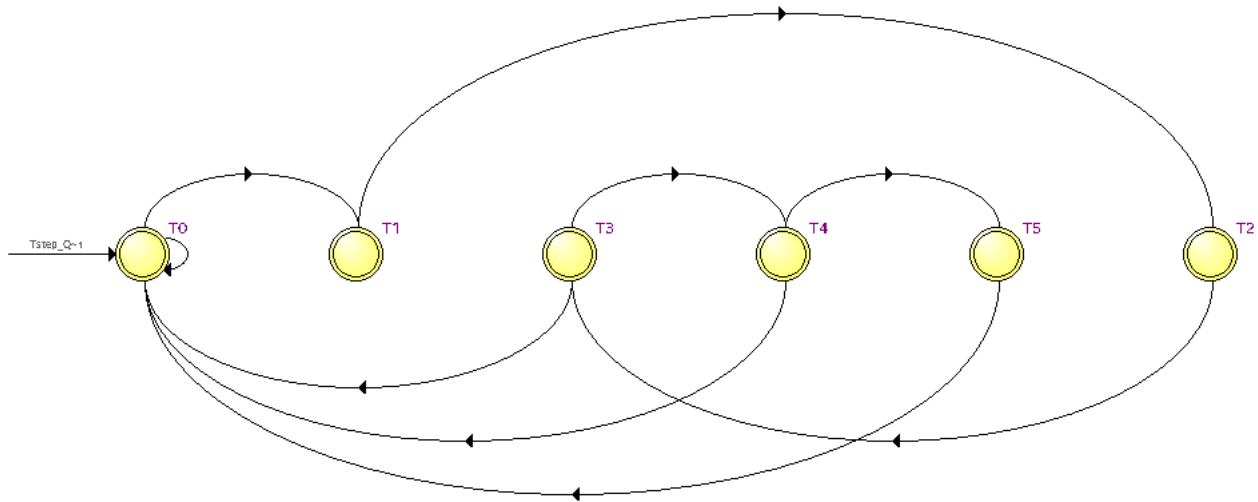


Tabela przejść

	Source State	Destination State	Condition
1	T0	T0	(!Run)
2	T0	T1	(Run)
3	T1	T2	
4	T2	T3	
5	T3	T0	(!Done)
6	T3	T4	(!Decoder0)
7	T4	T0	(!Decoder1)
8	T4	T5	(!Decoder1)
9	T5	T0	

Tabela kodowania

	Name	T5	T4	T3	T2	T1	T0
1	T0	0	0	0	0	0	0
2	T1	0	0	0	0	1	1
3	T2	0	0	0	1	0	1
4	T3	0	0	1	0	0	1
5	T4	0	1	0	0	0	1
6	T5	1	0	0	0	0	1

Wyniki symulacji funkcjonalnej

Zawartość pliku MIF z programem wykonywanym przez procesor:

```
WIDTH=9;
DEPTH=128;

ADDRESS_RADIX=UNS;
DATA_RADIX=BIN;

CONTENT BEGIN
    % adres 0, do którego wpisujemy "śmieci" - efekt uboczny instrukcji zapisu do rejestru HEX0
    (gdz ADDR[8:0] = 000000000) %
    0 : 000000000;
    % obszar do przechowywania kodów znaków %
    1 : 000010010; % S %
    2 : 000000110; % E %
    3 : 000100001; % d %
```

```

4 : 000100011; % o %
5 : 001000000; % mvi R0, #0 // zewnętrzny licznik po adresach rejestrów HEX %
6 : 000000000;
7 : 001001000; % mvi R1, #1 // delta - wartość, o jaką zmieniamy licznik po rejestrach; może
być tylko 1 (010000000) lub -1 (-010000000 = 110000000 w kodzie U2) %
8 : 010000000;
% start: %
9 : 001010000; % mvi R2, #4 // wewnętrzny licznik po znakach %
10 : 000000100;
11 : 000101111; % mv R5, R7 // zapisujemy adres następnej instrukcji %
12 : 100011010; % ld R3, [R2] // ładujemy znak, na który wskazuje licznik po znakach %
13 : 101011000; % st R3, [R0] // zapisujemy znak pod adresem wskazywanym przez licznik po
rejestrach %
14 : 001011000; % mvi R3, #010000000 // do inkrementacji licznika po rejestrach %
15 : 010000000;
16 : 010000011; % add R0, R3 // inkrementacja licznika po rejestrach %
17 : 001011000; % mvi R3, #1 // do dekrementacji licznika po znakach %
18 : 000000001;
19 : 011010011; % sub R2, R3 // dekrementacja licznika po znakach %
20 : 110111101; % mvnz R7, R5 // następna iteracja pętli po znakach %

% delay: funkcja opóźniająca; 511 * 511 * 8 iteracji %
21 : 001010000; % mvi R2, #1 // do dekrementacji opóźnień %
22 : 000000001;
23 : 001011000; % mvi R3, #8 // opóźnienie a; jeżeli nie chcemy go używać, to ustawiamy 1 i
wówczas pętla a skończy się po 1 iteracji %
24 : 000001000;
25 : 001100000; % a_loop: mvi R4, #511 // opóźnienie b %
26 : 111111111;
27 : 001101000; % b_loop: mvi R5, #511 // opóźnienie c %
28 : 111111111;
29 : 011101010; % c_loop: sub R5, R2 // dekrementacja opóźnienia c %
30 : 001110000; % mvi R6, #c_loop %
31 : 000011101;
32 : 110111110; % mvnz R7, R6 // następna iteracja pętli c, jeżeli opóźnienie c != 0 %
33 : 011100010; % sub R4, R2 // dekrementacja opóźnienia b %
34 : 001110000; % mvi R6, #b_loop %
35 : 000011011;
36 : 110111110; % mvnz R7, R6 // następna iteracja pętli b, jeżeli opóźnienie b != 0 %
37 : 011011010; % sub R3, R2 // dekrementacja opóźnienia a %
38 : 001110000; % mvi R6, #a_loop %
39 : 000011001;
40 : 110111110; % mvnz R7, R6 // następna iteracja pętli a, jeżeli opóźnienie a != 0 %

41 : 010000001; % add R0, R1 // inkrementacja licznika po rejestrach, aby pętla po znakach
zaczynała wpisywać od rejestru przesuniętego o 1 w prawo lub w lewo zależnie od wartości R1 %
42 : 001010000; % mvi R2, #start %
43 : 000001001;
44 : 110111010; % mvnz R7, R2 // wracamy do początku, jeżeli licznik po rejestrach nie jest
równy 0, czyli nie został przesunięty w prawo z 3 (110000000) na 0 lub w lewo z 1 (010000000) na
0 %

% w przeciwnym przypadku %
45 : 000010001; % mv R2, R1 // tworzymy kopię delty %
46 : 001011000; % mvi R3, #010000000 // 1 do odjęcia od kopii %
47 : 010000000;
48 : 011010011; % sub R2, R3 %
49 : 001010000; % mvi R2, #delta == -1 %
50 : 000111001;
51 : 110111010; % mvnz R7, R2 // jeżeli delta nie jest równa 1, to musi być równa -1, więc
zamieniamy ją na 1 %
% delta == 1: %
52 : 001001000; % mvi R1, #-1 // zmieniamy deltę na -1 (110000000) %

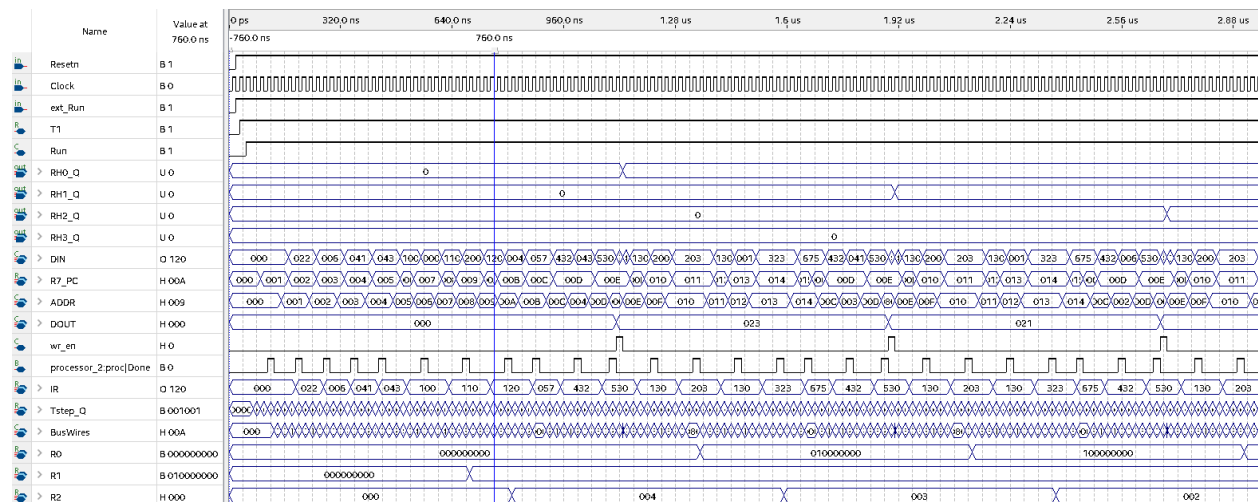
```

```

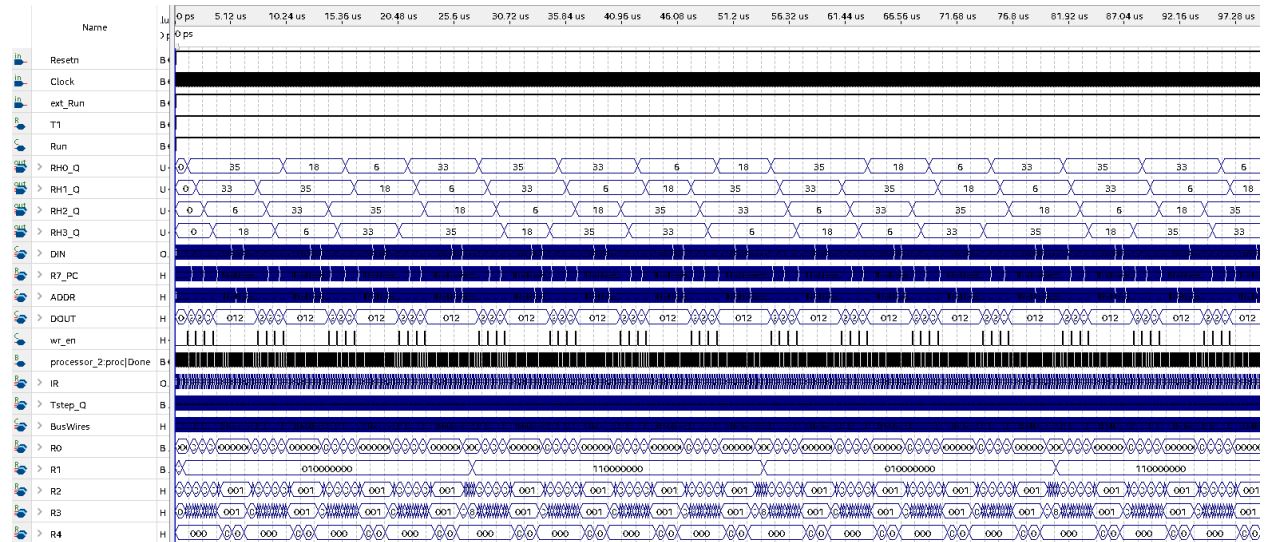
53 : 110000000;
54 : 001010000; % mvi R2, #start %
55 : 000001001;
56 : 000111010; % mv R7, R2 // wracamy do początku %
% delta == -1: %
57 : 001001000; % mvi R1, #1 // zmieniamy deltę na 1 (010000000) %
58 : 010000000;
59 : 001010000; % mvi R2, #start %
60 : 000001001;
61 : 000111010; % mv R7, R2 // wracamy do początku %
END;

```

Inicjujemy w rejestrze R0 zmienną wskazującą aktualny adres zewnętrznego rejestru, do którego wpisujemy kod znaku dla wyświetlacza 7-segmentowego. W rejestrze R1 inicjujemy zmienną o początkowej wartości 010000000, którą będziemy dodawać do R0, aby przesunąć znaki w prawo. Po osiągnięciu maksymalnego przesunięcia w prawo, ustawiamy w rejestrze R1 uzupełnienie do dwóch liczby 010000000, czyli 110000000. Następnie dalej dodajemy R1 do R0, co przesuną znaki w lewo. Po osiągnięciu maksymalnego przesunięcia w lewo, ponownie ustawiamy w R1 wartość 010000000, aby przesunąć znaki w prawo, itd.

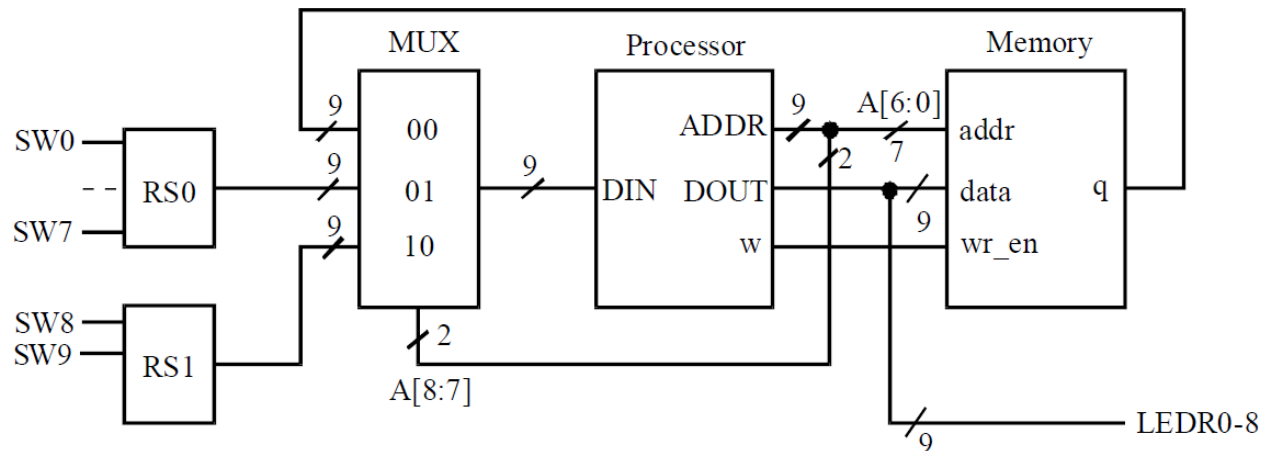


W zewnętrznych rejestrach RH0 - RH3 przesuwamy w prawo i w lewo kody dla wyświetlacza 7-segmentowego znaków o - 35, d - 33, E - 6, S - 18. Po każdym przesunięciu całego słowa, czyli 4 znaków, wykonujemy opóźnienie. Aby można było zobaczyć działanie układu, w symulacji stosujemy krótkie pętle opóźniające.



Zadanie 2

Utwórz projekt **processor_with_SW**, którego strukturę pokazano na rys. 5.



Rys. 5. Podłączanie rejestrów wejściowych do procesora

Ta struktura umożliwia procesorowi odczytanie przełączników SW na płycie. W tym celu do projektu dodane są zewnętrzne wejściowe rejestry 9-bitowe RS0 i RS1, których wejścia są bezpośrednio połączone z przełącznikami SW na płycie (SW0-8 są podłączone do RS0, SW9 do RS1). Linie adresowe A8 i A7 są wykorzystywane, aby wybrać źródło dla szyny DIN Wejście: 00 - Pamięć 01 - RS0, 10 - RS1.

Napisz program dla procesora, który sekwencyjnie skanuje przełączniki SW i wyprowadza ich wartość do diod LEDR. Symuluj projekt i testuj projekt na płycie.

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module processor_with_SW_on_board(  
    input [9:0] SW,  
    input CLOCK_50,  
    output [8:0] LEDR);  
  
    processor_with_SW proc(SW, 1'b1, CLOCK_50, 1'b1, LEDR);  
  
endmodule
```

Moduł procesora z pamięcią RAM i zewnętrznymi rejestrami wejściowymi

```
module processor_with_SW(  
    input [9:0] SW,  
    input Resetn, Clock, ext_Run,  
    output [8:0] LEDs);  
  
    wire [8:0] q;  
    reg [8:0] DIN;  
    wire [8:0] ADDR, DOUT /* synthesis keep */;  
    assign LEDs = DOUT;  
    wire wr_en /* synthesis keep */;  
    RAM128x9 ram(ADDR[6:0], Clock, DOUT, wr_en, q);  
    reg T1, T2 /* synthesis keep */;  
    always @ (posedge Clock) begin
```

```

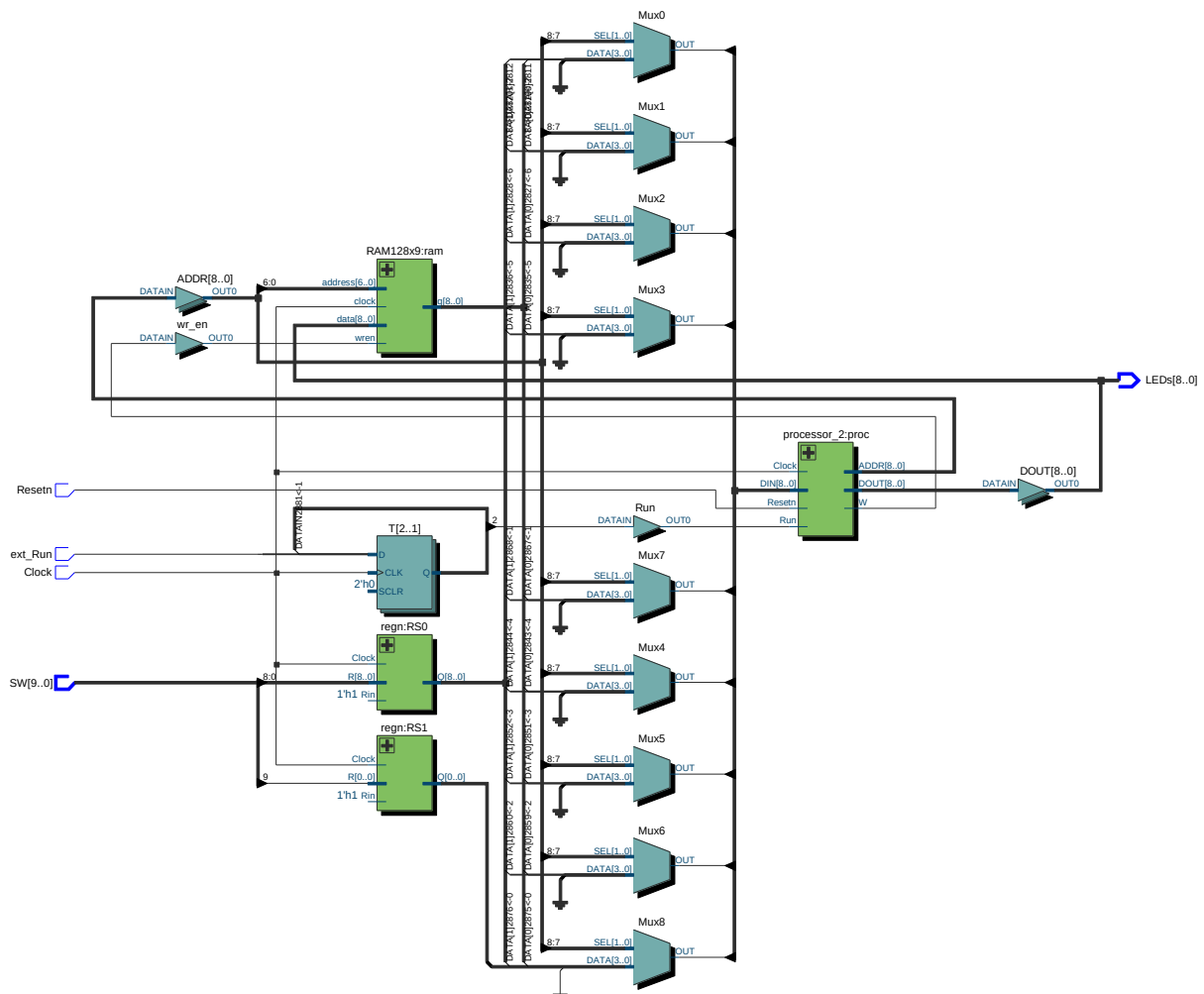
        T2 <= T1;
        T1 <= ext_Run;
    end
    wire Run = T2 /* synthesis keep */;
    processor_2 proc(DIN, Reseth, Clock, Run, ADDR, DOUT, wr_en);
    // multiplekser
    wire [8:0] RS0_Q;
    wire RS1_Q;
    always @ (*)
        case (ADDR[8:7])
            2'b00: DIN = q;
            2'b01: DIN = RS0_Q;
            2'b10: DIN = { 8'b00000000, RS1_Q };
            default: DIN = 9'b000000000;
        endcase
    regn #(9) RS0(SW[8:0], 1'b1, Clock, RS0_Q);
    regn #(1) RS1(SW[9], 1'b1, Clock, RS1_Q);

endmodule

```

Pozostałe moduły: processor_2, regn, dec3to8, ripple_carry_adder_subtractor, full_adder są takie same jak w zadaniu 1.

Wyniki syntezy



Wyniki symulacji funkcjonalnej

Zawartość pliku MIF z programem wykonywanym przez procesor:

```
WIDTH=9;
DEPTH=128;

ADDRESS_RADIX=UNS;
DATA_RADIX=BIN;

CONTENT BEGIN
    % adres 0, do którego wpisujemy "śmieci" - efekt uboczny instrukcji zapisu do DOUT (do LEDów),
    gdy ADDR[8:0] = 000000000 %
    0 : 000000000;
    % start: %
    1 : 001000000; % mvi R0, #010000000 // adres RS0 %
    2 : 010000000;
    3 : 100001000; % ld R1, [R0] // ładujemy wartość rejestru RS0 %
    4 : 001000000; % mvi R0, #000000000 // adres 0, aby wpisać słowo z RS0 do DOUT i nie nadpisać
    żadnego adresu z kodem %
    5 : 000000000;
    6 : 101001000; % st R1, [R0] // zapisujemy wartość rejestru RS0 w DOUT %

    7 : 001000000; % mvi R0, #2 // zapisujemy w R0 przesunięcie, do którego dodamy adres
    wywołania, aby przeskoczyć wywołanie funkcji; dodajemy 2, ponieważ chcemy przeskoczyć adresy "mvi
    R7, #delay" i "delay" %
    8 : 000000010;
    9 : 010000111; % add R0, R7 // obliczamy adres następnej instrukcji + 2, aby uzyskać adres
    powrotu z funkcji opóźniającej %
    10 : 001111000; % mvi R7, #delay // skok do funkcji opóźniającej %
    11 : 000011001;

    % adres powrotu z pierwszego wywołania funkcji opóźniającej: %
    12 : 001000000; % mvi R0, #100000000 // adres RS1 %
    13 : 100000000;
    14 : 100001000; % ld R1, [R0] // ładujemy wartość rejestru RS1 %
    15 : 001000000; % mvi R0, #000000000 // adres 0, aby wpisać słowo z RS1 do DOUT i nie nadpisać
    żadnego adresu z kodem %
    16 : 000000000;
    17 : 101001000; % st R1, [R0] // zapisujemy wartość rejestru RS1 w DOUT %

    18 : 001000000; % mvi R0, #2 // zapisujemy w R0 przesunięcie, do którego dodamy adres
    wywołania, aby przeskoczyć wywołanie funkcji; dodajemy 2, ponieważ chcemy przeskoczyć adresy "mvi
    R7, #delay" i "delay" %
    19 : 000000010;
    20 : 010000111; % add R0, R7 // obliczamy adres następnej instrukcji + 2, aby uzyskać adres
    powrotu z funkcji opóźniającej %
    21 : 001111000; % mvi R7, #delay // skok do funkcji opóźniającej %
    22 : 000011001;

    % adres powrotu z drugiego wywołania funkcji opóźniającej: %
    23 : 001111000; % mvi R7, #start // wracamy do początku %
    24 : 000000001;

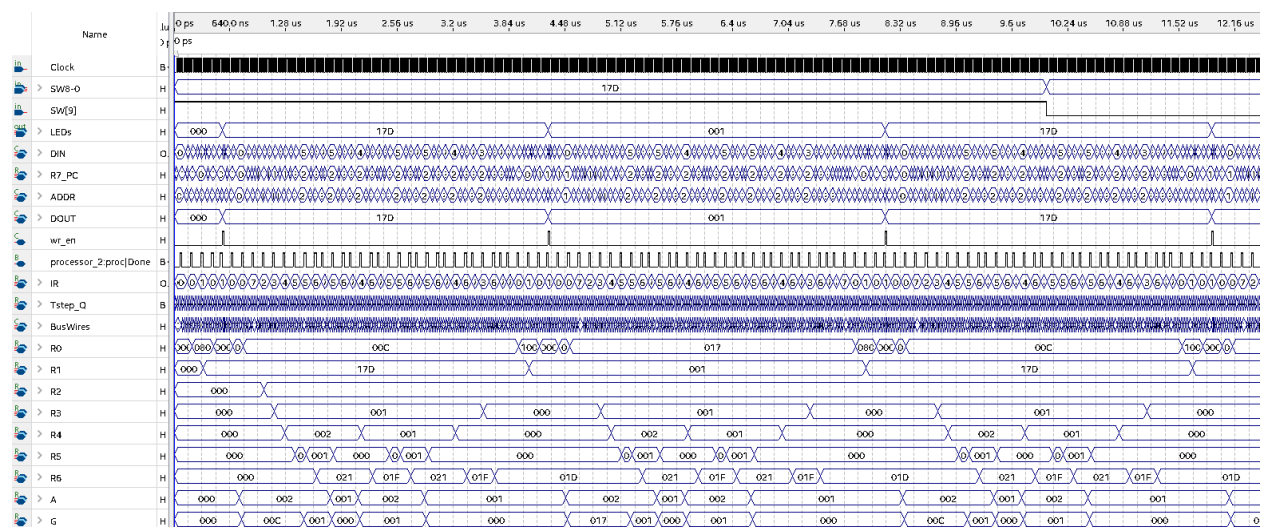
    % delay: funkcja opóźniająca; 511 * 511 * 8 iteracji; adres powrotu musi być w R0 %
    25 : 001010000; % mvi R2, #1 // do dekrementacji opóźnień %
    26 : 000000001;
    27 : 001011000; % mvi R3, #8 // opóźnienie a; jeżeli nie chcemy go używać, to ustawiamy 1 i
    wówczas pętla a skończy się po 1 iteracji %
    28 : 000001000;
    29 : 001100000; % a_loop: mvi R4, #511 // opóźnienie b %
    30 : 111111111;
```

```

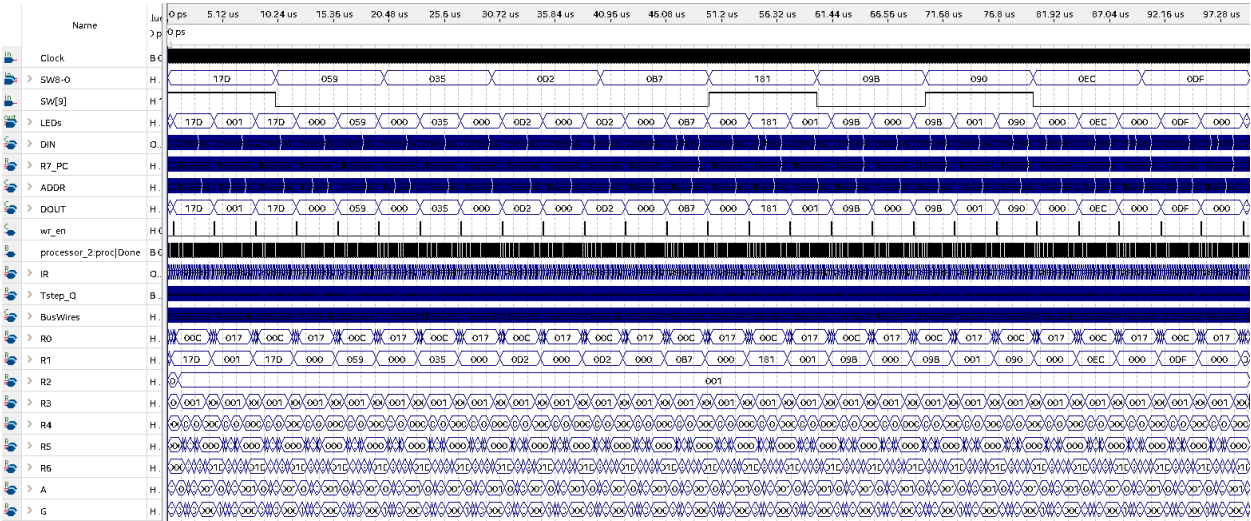
31 : 001101000; % b_loop: mvi R5, #511 // opóźnienie c %
32 : 111111111;
33 : 011101010; % c_loop: sub R5, R2 // dekrementacja opóźnienia c %
34 : 001110000; % mvi R6, #c_loop %
35 : 000100001;
36 : 110111110; % mvnz R7, R6 // następna iteracja pętli c, jeżeli opóźnienie c != 0 %
37 : 011100010; % sub R4, R2 // dekrementacja opóźnienia b %
38 : 001110000; % mvi R6, #b_loop %
39 : 000011111;
40 : 110111110; % mvnz R7, R6 // następna iteracja pętli b, jeżeli opóźnienie b != 0 %
41 : 011101010; % sub R3, R2 // dekrementacja opóźnienia a %
42 : 001110000; % mvi R6, #a_loop %
43 : 000011101;
44 : 110111110; % mvnz R7, R6 // następna iteracja pętli a, jeżeli opóźnienie a != 0 %
45 : 000111000; % mv R7, R0 // skok do adresu powrotu %
END;

```

W każdym cyklu ładujemy do rejestru R1 9-bitową wartość zewnętrznego rejestru RS0 (SW8-0) i zapisujemy wartość R1 do rejestru DOUT, który podaje ją na LEDy. Po tym wykonujemy 3 zagnieżdżone pętle opóźniające, których liczniki przechowujemy w rejestrach R3, R4, R5. Po opóźnieniu ładujemy do R1 1-bitową wartość RS1 (SW9) i zapisujemy wartość R1 do DOUT. Tym sposobem widzimy na LEDach najpierw wartość SW8-0, a następnie SW9, później znów SW8-0 itd.



Aby można było zobaczyć działanie układu, w symulacji stosujemy krótkie pętle opóźniające.



Wnioski

Wszystkie zadania zostały zrealizowane.